SAR: State Aware Retrieval for Iterative Retrieval Augmented Generation

A Project Report Submitted in

partial fulfillment of requirements for the degree of

Bachelor of Technology

In

Computer Engineering

By

Kunal Dahal (22CSEC17)

Aniket Manger (22CSEC23)

Suman Chettri (22CSEC44)

Under the supervision of

Mr. Anmol Pandey

Assistant Professor

Computer Engineering Department



Department Of Computer Engineering

Sikkim Institute of Science and Technology

(A constituent college of Sikkim University)

2026

ABSTRACT

Retrieval Augmented Generation (RAG) uses external documents to help large language models answer complex questions. However, during iterative retrieval, documents that are relevant to the original question may repeat information that has already been retrieved, while other required information is still missing. This leads to redundant retrieval and incomplete evidence collection.

This project proposes **State Aware Retrieval (SAR)** for iterative RAG. SAR uses a Sentence Transformer to represent the question, information gaps, collected evidence, and documents in a common semantic space. Based on these representations, a state aware scoring equation assigns a score to every document. The equation combines three factors: relevance to the original question, coverage of the current information gap, and novelty with respect to the evidence already collected. This allows retrieval to consider both what is relevant to the question and what information is still required. The documents are then ranked by their final scores, and the Top-K documents with the highest scores are selected for the next retrieval step. SAR therefore selects documents according to the current retrieval state instead of relying only on the original question.

The proposed approach aims to reduce redundant retrieval and to select more complementary evidence for complex question answering. Its effectiveness will be studied through a systematic evaluation of retrieval quality and answer quality.

Keywords: Retrieval Augmented Generation, Information Retrieval, Iterative Retrieval, Multi-Hop Question Answering.

TABLE OF CONTENTS

CHP NO.	TOPICS	PAGE NO.
	ABSTRACT	i
	TABLE OF CONTENTS	ii
	LIST OF TABLES	iii
	LIST OF FIGURES	iv
1.	INTRODUCTION	1
2.	LITERATURE SURVEY	2–6
3.	PROBLEM DEFINITION	7
4.	OBJECTIVES	8
5.	METHODOLOGY	9–22
6.	SOFTWARE AND HARDWARE REQUIREMENTS	23–24
7.	IMPLEMENTATION	25–34
8.	APPLICATIONS	35
9.	GANTT CHART	36
10.	CONCLUSION	37
	REFERENCES	38

LIST OF TABLES

TABLE NO.	TITLE	PAGE NO.
2.1	Literature Survey 1	2
2.2	Literature Survey 2	3
2.3	Literature Survey 3	4
2.4	Literature Survey 4	5
2.5	Literature Survey 5	6
5.1	State Aware Retrieval Configuration	16
5.2	State Aware Retrieval Hyperparameters	16
5.3	Models used in SAR, MIGRES, and baseline systems	18
5.4	Information Retrieval Baseline Comparison	20
6.1	Software Requirements	23
6.2	Modules Description	??
7.1	Hardware Environment	25
7.2	Software Environment	26
7.3	Experiment Notebooks and Their Mapping to the Experimental Design	33
7.4	Outputs Saved for Each Experimental Run	34

LIST OF FIGURES

FIGURE NO.	TITLE	PAGE NO.
5.1	Architecture of State Aware Retrieval	10
5.2	Operational Flow of State Aware Retrieval	11
7.1	Overall System Implementation Flow	26
7.2	Corpus Preparation and Embedding Pipeline	27
7.3	MIGRES State and Processing Flow with SAR Integration	27
7.4	Architecture of the SAR Retrieval Module	28
7.5	Project Structure of the SAR–MIGRES Implementation	29
7.6	Code-Level Architecture of the SAR–MIGRES Implementation	30
7.7	Implementation of the SAR Scoring and Retrieval Module	31
9.1	Gantt Chart	36

CHAPTER 1

INTRODUCTION

Retrieval Augmented Generation (RAG) is a technique that combines information retrieval with large language models to improve question answering. Instead of depending only on the information stored inside a language model, RAG retrieves relevant information from external documents and uses it to generate an answer [1]. This makes RAG useful for tasks where the required information is spread across different documents.

However, answering complex questions is not as simple as finding documents that are related to the question. Some questions need several pieces of information, and the first retrieval step may provide only part of what is required. Multi-hop question answering benchmarks and iterative retrieval methods show that such questions can require evidence from multiple passages or documents and repeated retrieval steps [2, 3, 4]. If the retrieval system keeps selecting documents mainly because they are related to the original question, it may retrieve similar information again and again instead of finding the missing evidence.

MIGRES (Missing Information Guided Retrieve-Extraction-Solving) provides a framework for handling this type of iterative information retrieval. It keeps track of the information already collected and identifies the information that is still required to answer the question [5]. This creates an opportunity to improve the retrieval stage by considering not only the original question, but also the current information gap and the evidence already collected. Related work on relevant information gain also supports reducing redundant evidence while preserving useful information for RAG [6].

This project proposes State Aware Retrieval (SAR), a retrieval method that improves document selection in such iterative RAG systems. The main idea is to select documents based on three things: their relevance to the question, their usefulness for covering the missing information, and the amount of new information they provide compared with the evidence already collected.

The proposed approach will be studied within the MIGRES framework and evaluated on question-answering datasets. The overall aim of this project is to examine whether making the retrieval process aware of the current information state can help RAG systems retrieve more useful and complementary evidence for complex questions.

CHAPTER 2

LITERATURE SURVEY

Table 2.1: Literature Survey 1

Name of paper	Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks [1]
Authors	Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela
Publication Details	Advances in Neural Information Processing Systems 33 (NeurIPS), 2020, pp. 9459–9474.
Research Findings	• RAG combines information stored in a language model with information retrieved from external documents. • Retrieved passages help the model answer questions that require outside knowledge. • The approach shows that retrieval can improve the performance of language generation for knowledge-intensive tasks. • The work provides a basic foundation for modern retrieval-augmented generation systems.
Research Gap	• RAG searches for documents only once, using the input question alone. It cannot search again after reading the first results. • On some tasks the retriever stops learning anything useful and returns the same documents for every input. The generator then learns to ignore the documents. • Retrieving more documents does not always help. For the RAG-Token model, accuracy stops improving at about 10 documents. • The paper tests only single-step tasks such as NQ, TriviaQA and FEVER. It includes no multi-hop question answering benchmark, so questions that need several linked facts are not studied.
Relevance with Our Work	• This paper gives the basic retrieve-then-generate setup that our work starts from. • Because RAG searches once from the question alone, it cannot look for evidence that only becomes needed after earlier evidence has been read. Our work retrieves in several steps, and each step uses the current information state.

Table 2.2: Literature Survey 2

Name of paper	Leveraging Passage Retrieval with Generative Models for Open Domain Question Answering [7]
Authors	Gautier Izacard and Edouard Grave
Publication Details	Proceedings of the 16th Conference of the European Chapter of the Association for Computational Linguistics (EACL), 2021, pp. 874–880.
Research Findings	• The method retrieves multiple passages and provides them to a generative model. • The model can use information from several passages to answer a question. • Using useful information from multiple retrieved passages can improve question-answering performance. • The work shows the importance of selecting and using several relevant pieces of evidence.
Research Gap	• The passages come from a separate retriever (BM25 or DPR [8]) that runs once per question. The reader cannot ask for better passages, and the authors list joint training of retrieval and reading as future work. • Accuracy keeps rising as passages are added, up to 100. Going from 10 to 100 passages gives about 6% more on TriviaQA and 3.5% more on NQ. This shows that the top few passages often do not cover all the needed evidence. • Reading many passages is costly. Training with fewer passages lowers accuracy, and the authors name efficiency with large numbers of passages as future work. • The paper tests only single-hop question answering (NQ, TriviaQA and SQuAD), not multi-step questions.
Relevance with Our Work	• This paper shows that a model answers better when it can combine evidence from several passages. • Its way of improving coverage is to read many more passages, which is expensive. Our work instead selects a small set of passages, where each one adds evidence that is still missing.

Table 2.3: Literature Survey 3

Name of paper	LLMs Know What They Need: Leveraging a Missing Information Guided Framework to Empower Retrieval-Augmented Generation [5]
Authors	Keheng Wang, Feiyu Duan, Peiguang Li, Sirui Wang, and Xunliang Cai
Publication Details	Proceedings of the 31st International Conference on Computational Linguistics (COLING), 2025, pp. 2379–2400.
Research Findings	• MIGRES identifies information that is still missing after processing the retrieved evidence. • The identified missing information is used to guide the next retrieval step. • The framework performs retrieval and evidence processing in multiple steps. • The approach is designed for questions that require several pieces of information.
Research Gap	• The authors found passages that look very relevant to the query but give no useful information, and they had to add a memory module to remove them. A stronger dense retriever also worked slightly worse than BM25 [9], because it returned passages on the same topic that lacked the needed fact. Together these show that relevance alone is not a reliable way to choose evidence. • Passages are kept or removed using a relevance threshold against the generated queries. The choice does not consider how much new information a passage adds to the evidence already collected. • The method uses many LLM calls: 6.4 to 9.5 per question on multi-hop datasets, compared with 2 for ITER-RETGEN. • When no passage passes the threshold, the LLM writes the knowledge itself. With GPT-3.5, this text contains the right sub-answer only 45.5%, 54.6% and 24.7% of the time on Wikihop, Musique and NQ.
Relevance with Our Work	• This paper gives the main framework we build on: find the missing information, then use it to guide the next retrieval step. • We address its weak selection step. Instead of relevance and a fixed threshold alone, we rank candidate documents by how well they match the missing information and how much new content they add beyond the evidence already collected.

Table 2.4: Literature Survey 4

Name of paper	Unsupervised Alignment-based Iterative Evidence Retrieval for Multi-hop Question Answering [4]
Authors	Vikas Yadav, Steven Bethard, and Mihai Surdeanu
Publication Details	Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL), 2020, pp. 4514–4525.
Research Findings	• AIR is an iterative method designed to retrieve evidence for multi-hop questions. • The retrieval process changes after each step instead of using the same query throughout. • Later retrieval focuses on information that is not already covered by the available evidence. • The work shows that iterative retrieval can help collect complementary evidence.
Research Gap	• AIR decides what is covered using word-level GloVe similarity with a fixed threshold (0.95). A query word counts as covered if a similar word appears in a retrieved sentence, but this does not prove that the needed fact is actually there. The method also depends on hand-set thresholds, and a larger query-expansion threshold lowers F1 on MultiRC. • Retrieval is greedy. Each step takes the top sentence for the current query, and there is no separate step that compares candidates with the evidence already collected. • Adding parallel evidence chains improves answer accuracy but lowers evidence precision. F1 falls from 64.2 to 44.9 with five chains, and more steps are needed. • The query is built from the question plus a candidate answer, so the method is designed for multiple-choice questions. Open-ended questions are not tested.
Relevance with Our Work	• This paper shows that later retrieval should focus on information that is not yet covered, and that doing so reduces semantic drift. • We address its word-level coverage check and greedy selection. Our work scores whole candidate documents against the missing information and the evidence already collected.

Table 2.5: Literature Survey 5

Name of paper	Better RAG using Relevant Information Gain [6]
Authors	Marc Pickett, Jeremy Hartman, Ayan Kumar Bhowmick, Raquib-ul Alam, and Aditya Vempaty
Publication Details	arXiv preprint, 2024, arXiv:2407.12101.
Research Findings	• The paper studies how relevant information gain can be used to improve document selection in RAG. • It aims to reduce redundant information while keeping the retrieved information relevant. • The approach gives importance to documents that provide useful information not already present in the selected evidence. • The work highlights the importance of balancing relevance with the usefulness of new information.
Research Gap	• The information gain is measured against the original query, and all k passages are chosen in one step. The selection does not change as new evidence is read. • On questions that need several passages, the best NDCG is 0.609 while the oracle reaches 0.826, and end-to-end accuracy is at most 42%. This shows that single-step selection struggles with multi-part questions. • The method needs a hand-tuned parameter (σ) that controls how much diversity is encouraged. • The experiments use one benchmark (RGB [10]), one LLM (ChatGLM) and exact-string-match scoring. The authors say it is not yet known whether the results hold elsewhere.
Relevance with Our Work	• This paper supports the idea that selection should reduce repeated content and not rely on relevance alone. • Its gain is tied to the fixed original query, which works poorly for questions with several parts. Our work recomputes the selection at every step, using the current missing information and the evidence already collected.

CHAPTER 3

PROBLEM DEFINITION

1. Documents are mainly selected by their similarity to the query, so later retrieval steps keep returning documents similar to those already found.
2. The information that is still missing is not used in document selection, so documents that can fill the gap are not given priority.
3. Previously collected evidence is not used when ranking documents, so redundant documents are selected while complementary ones are missed.
4. As a result, the selected evidence can look relevant but stay incomplete, and the model may not get what it needs to answer the question.

CHAPTER 4

OBJECTIVES

1. To keep the selected documents related to the query without depending on it alone, so that later retrieval steps do not keep returning documents similar to those already found.
2. To give priority to documents that provide the missing information, so that documents able to fill the gap are ranked higher.
3. To lower the rank of documents that repeat the evidence already collected, so that complementary documents are selected instead of redundant ones.
4. To propose a scoring approach that combines relevance, gap coverage, and novelty to rank documents at each retrieval step, and to evaluate it through experiments.

CHAPTER 5
METHODOLOGY

5.1 Solution Strategy

1. Every document is converted into a vector once using a Sentence Transformer model [11] and saved on disk. At each step, the original question is converted in the same way and compared with every document. Closer documents get a higher score, so the selection stays related to what was asked without relying on the question alone.

2. MIGRES [5] creates small queries for the information that is still missing. Each query is converted into a vector and compared with every document, and the highest score across the queries is kept for each document. Closer documents get a higher score. In the first step there are no gaps, so this score is not used. This gives priority to documents that fill the gap.

3. The useful information already extracted by MIGRES is treated as the collected evidence. A document gets a lower score when it is very similar to any item of this evidence, and a higher score when it brings something new. In the first step every document is treated as new. This prefers complementary documents over redundant ones.

4. The three scores are combined using weights to give one final score per document. The weights are chosen on validation data using Bayesian optimization [12] and fixed before testing. All documents are scored at every step, and the top-K documents are sent to MIGRES, which updates the evidence and gaps.

5.2 Proposed Solution

In iterative retrieval, a document may be relevant to the original question but may not give the information that is still missing. It may also repeat information that has already been collected. This leads to repeated retrieval and less useful evidence [4, 6, 5].

To address this, we propose **State Aware Retrieval (SAR)**, a retrieval method that replaces the document retrieval step inside MIGRES [5]. At each iteration, SAR uses three things: the original question q, the information gaps G_t that are still missing, and the evidence E_t collected so far.

SAR converts the documents, question, gaps, and evidence into vectors using the same Sentence Transformer model. A document scores higher if it is related to the question and close to the missing information, and lower if it repeats the collected evidence. These three scores are combined into one final score using weights tuned on validation data, and the top-K documents are selected.

The selected documents are passed to MIGRES, which filters them, extracts evidence, and updates the gaps. SAR then uses the updated state in the next iteration. In the first iteration there are no gaps and no evidence, so the ranking depends only on the question.

5.2.1 Architecture Diagram

Figure 5.1 shows how SAR works inside MIGRES: MIGRES provides the question, gaps, and evidence at each iteration, and SAR returns the top-K ranked documents.

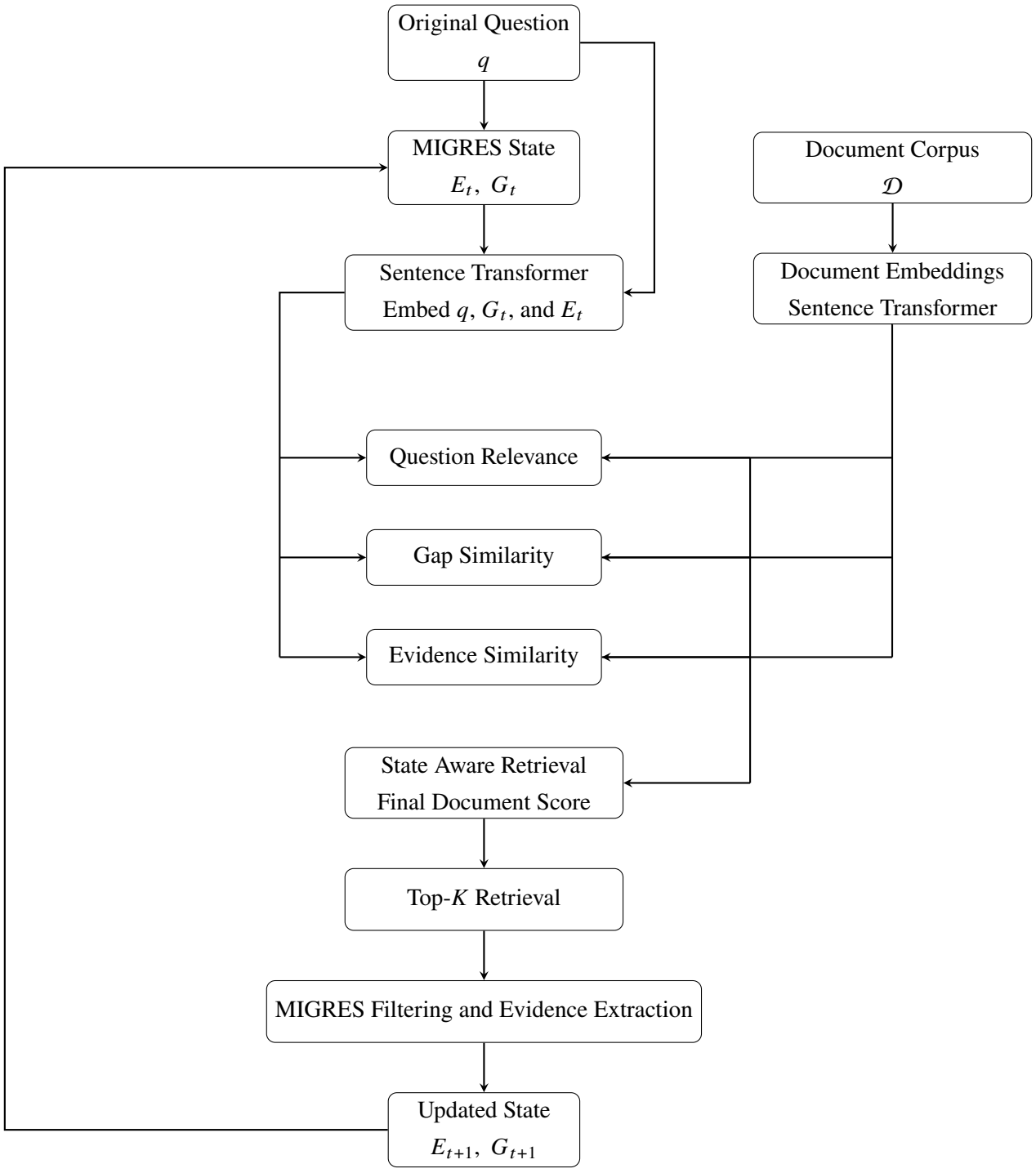


Figure 5.1: Architecture of State Aware Retrieval

5.2.2 Operational Flow

The operational flow describes how the proposed system performs retrieval at each iteration. MIGRES provides the current question, evidence, and information gaps. These inputs are embedded and compared with the document corpus. State Aware Retrieval then scores and selects the top-K documents, which are passed to MIGRES for filtering, evidence extraction, and state update. The updated state is used in the next iteration.

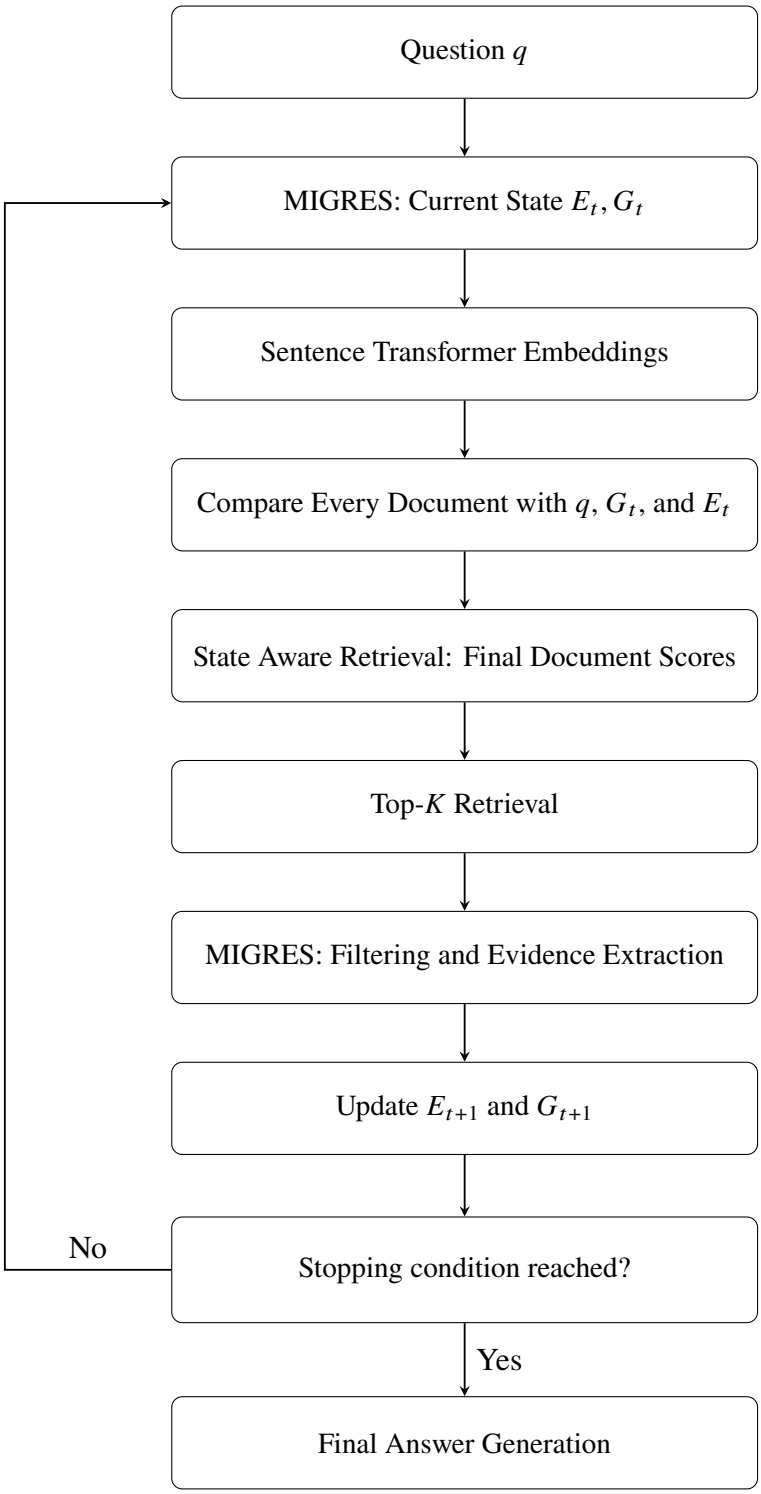


Figure 5.2: Operational Flow of State Aware Retrieval

5.3 Mathematical Formulation

At iteration t, the retrieval state contains the question q, the collected evidence E_t, the current information gap G_t, and the complete corpus $\mathcal{D}$. The retrieval method uses this state to assign one final score to every document in the corpus.

5.3.1 Notation

Let

$$E_t = \{e_1, e_2, \ldots, e_n\}$$

be the evidence collected before iteration t, and let

$$G_t = \{g_1, g_2, \ldots, g_m\}$$

be the information gaps identified by MIGRES. The searchable document corpus is:

$$\mathcal{D} = \{d_1, d_2, \ldots, d_N\}.$$

The current retrieval state is represented as:

$$\boxed{(q, E_t, G_t, \mathcal{D})}.$$

The first retrieval iteration is indexed $t = 0$, and at this point there is no evidence and no gap, so $E_0 = \emptyset$ and $G_0 = \emptyset$. In the implementation, E_t is the list of useful information that MIGRES has extracted so far, and G_t is the list of sub-queries that MIGRES creates for the information that is still missing.

5.3.2 Corpus Embedding

Before retrieval begins, all documents in the corpus are encoded using the selected Sentence Transformer model. The resulting document embeddings are stored and reused during retrieval:

$$\phi(d) \leftarrow \text{Sentence Transformer}(d).$$

At the end of each retrieval iteration, the system checks whether any documents in the corpus are new or changed. A new document is converted into a vector with the same model and added to the stored vectors. A changed document is converted again and its old vector is replaced. Documents that have not changed are left as they are, so nothing is converted twice.

5.3.3 State Embedding

At each iteration, the question, current gaps, and collected evidence are encoded using the same Sentence Transformer:

$$\phi(q), \qquad \phi(g_i), \qquad \phi(e_j).$$

This places the question, gaps, evidence, and documents in the same semantic space.

5.3.4 Sentence Similarity

The semantic similarity between two text items x and y is measured using cosine similarity, which is commonly used to compare sentence embeddings [11]:

$$\cos(x, y) = \frac{\phi(x)^T \phi(y)}{\|\phi(x)\|_2 \|\phi(y)\|_2}.$$

For the proposed scoring method, the value is normalized to the range $[0, 1]$ using:

$$\boxed{s(x, y) = \frac{1 + \cos(x, y)}{2}} \tag{1}$$

A higher value means that the two representations are more semantically similar.

5.3.5 Question Relevance

Question relevance measures how close a document is to the original question q. It uses the similarity $s(d, q)$ from Eq. 1, which is a value between 0 and 1:

$$\boxed{R(d, q) = s(d, q)} \tag{2}$$

Here $R(d, q)$ is the question relevance score of document d. A higher value means the document is more related to the main question. This signal keeps the retrieved documents on the topic of the question. It is the same kind of signal that dense retrievers use to rank passages for a question [8].

5.3.6 Gap Coverage

At iteration t, MIGRES provides m information gaps $G_t = \{g_1, \ldots, g_m\}$, where each g_i is a sub-query for one piece of missing information. First, the document d is compared with each gap separately:

$$\boxed{c(d, g_i) = s(d, g_i)} \tag{3}$$

Here $c(d, g_i)$ is the score of document d for the single gap g_i. A high value means the document is close to that missing piece.

Next, the scores for all gaps are averaged into one value:

$$\boxed{C(d, G_t) = \frac{1}{m} \sum_{i=1}^{m} c(d, g_i)} \tag{4}$$

A higher $C(d, G_t)$ means the document is closer to the information that is still missing. At the first iteration there are no gaps ($G_0 = \emptyset$), so the gap coverage is set to $C(d, G_0) = 0$. If no gap is left after an iteration, MIGRES can stop the retrieval.

5.3.7 Evidence Novelty

Evidence novelty measures how much new information a document brings compared with the evidence $E_t = \{e_1, \ldots, e_n\}$ already collected by MIGRES. First, the document d is compared with every evidence item e_j using $s(d, e_j)$, and the highest value is kept. This highest value shows how much the document repeats its closest piece of evidence. The novelty score is then one minus this value:

$$\boxed{N(d, E_t) = 1 - \max_{e_j \in E_t} s(d, e_j)} \tag{5}$$

A document that is very similar to some collected evidence has a high maximum similarity, so it receives a low novelty value. A document that is not similar to any collected evidence receives a high novelty value. This follows the idea of diversity-aware ranking, where a document is penalized for being similar to items that were already selected [13, 14], but here the comparison is made with the collected evidence. At the first iteration no evidence has been collected, so every document is treated as fully new:

$$\boxed{N(d, E_0) = 1.} \tag{6}$$

5.3.8 State Aware Document Score

The three signals are combined into one final score for every document. Each signal is multiplied by a weight and the results are added:

$$\boxed{S_t(d \mid q, E_t, G_t) = \alpha R(d, q) + \beta C(d, G_t) + \gamma N(d, E_t)} \tag{7}$$

The weights are never negative and together they add up to one:

$$\boxed{\alpha, \beta, \gamma \geq 0, \qquad \alpha + \beta + \gamma = 1.} \tag{8}$$

Here, α controls how much question relevance matters, β controls how much gap coverage matters, and γ controls how much novelty matters. The weights are needed because the three signals work in different directions. Relevance and gap coverage reward documents that are similar to the question or the missing information, while novelty rewards documents that are different from the collected evidence. An unrelated document is very different from the evidence and can therefore receive a high novelty score. The weights balance the signals so that novelty does not outweigh relevance. At the first iteration, $C = 0$ and $N = 1$ for every document, so only $R(d, q)$ changes the order of the documents.

5.3.9 Top-K Retrieval

After the final score S_t is calculated for every document in the corpus $\mathcal{D}$, the documents are ranked from the highest score to the lowest. The K documents with the highest scores form the retrieval result:

$$\boxed{T_t = \mathrm{TopK}_{d \in \mathcal{D}} S_t(d \mid q, E_t, G_t)} \tag{9}$$

Here T_t is the set of documents selected at iteration t, and K is the number of documents returned.

No fixed similarity threshold is used before the Top-K selection, so the K best-scored documents are always returned.

5.3.10 Iterative State Update

The selected documents T_t are passed to MIGRES for filtering and information extraction. The useful extracted information is added to the evidence state, and MIGRES identifies the remaining gap for the next iteration:

$$\boxed{(q, E_t, G_t, \mathcal{D}) \rightarrow T_t \rightarrow (E_{t+1}, G_{t+1}) \rightarrow T_{t+1}} \tag{10}$$

The process continues until MIGRES determines that enough information has been collected or the MIGRES iteration limit (5 by default) is reached.

5.4 State Aware Retrieval Configurations

Table 5.1 lists the main configuration of the proposed State Aware Retrieval system. It specifies the models, similarity method, retrieval space, document embedding setup, and other tools used in the project.

Table 5.1: State Aware Retrieval Configuration

Setting	**Value**	**Purpose**
Embedding model	multi-qa-mpnet-base-cos-v1	Creates common semantic representations for questions, documents, gaps, and evidence.
Similarity function	Cosine similarity	Measures semantic similarity between embedding vectors.
Retrieval space	Complete document corpus $\mathcal{D}$	Every document is considered during State Aware Retrieval.
Document embeddings	Precomputed and maintained	Stored once and updated when new or changed documents appear.
Framework	MIGRES	Provides the iterative evidence and information-gap state.
LLM API	gemini-3.5-flash-lite	Language model used by MIGRES, called through the Gemini API in the same request format as the original MIGRES code.

The weights α, β, and γ (Table 5.2) follow the constraint in Eq. 8 and are fixed before the final test evaluation. The value of K (50) follows the candidate pool size of the original MIGRES retriever. The iteration limit (5) and the stopping rule are also taken from MIGRES, so all three are the same for SAR and every baseline. SAR and every baseline also use the same language model and call settings, with the default temperature of the provider.

Table 5.2: State Aware Retrieval Hyperparameters

Hyperparameter	**Role**	**Selection**
α	Question relevance weight	Bayesian optimization on validation data.
β	Gap coverage weight	Bayesian optimization on validation data.
γ	Evidence novelty weight	Bayesian optimization on validation data.

5.5 Datasets and Model

5.5.1 Datasets

The evaluation uses three question-answering datasets from the MIGRES setting. They cover multi-hop reasoning and open-domain question answering.

HotpotQA

HotpotQA [2] has about 113,000 crowd-written questions over Wikipedia. Each question needs facts from two or more paragraphs, either as a *bridge* (one fact leads to the next) or a *comparison*. The supporting sentences are marked, so the retrieved evidence can be checked. In the distractor setting, each question has 10 paragraphs (2 correct, 8 distractors), and in the full-wiki setting the evidence is searched from all of Wikipedia.

Every question is stored as one record with fields such as the following:

Field	**Example**
`question`	Were Scott Derrickson and Ed Wood of the same nationality?
`answer`	yes
`supporting_facts`	Page titles and sentence numbers of the proof sentences
`context`	The Wikipedia paragraphs given for the question
`type`	comparison (or bridge)
`level`	easy, medium, or hard

MuSiQue

MuSiQue [3] has about 25,000 questions that need two to four reasoning steps, where the answer to one step is used in the next. The questions are built by joining simple one-step questions, and each comes with 20 Wikipedia paragraphs, most of them distractors. The design prevents a model from answering through shortcuts.

Every question is stored as one record with fields such as the following:

Field	**Content**
`question`	The full multi-step question
`question_decomposition`	The smaller one-step questions, in order
`paragraphs`	The 20 paragraphs, each with a title, text, and a mark showing if it is a correct (supporting) paragraph
`answer`	The short final answer
`answer_aliases`	Other accepted ways to write the answer
`answerable`	True or false

Natural Questions

Natural Questions [15] contains real, short queries typed into Google, with Wikipedia pages as the source. Annotators marked the answer, or marked that the page has none. Most questions need only

one passage, so it tests standard open-domain question answering. It has more than 300,000 training questions.

Every question is stored as one record with fields such as the following:

Field	Content
question_text	The search query, e.g., who sings what a wonderful world
document	The Wikipedia page found for the question
long_answer	The paragraph that contains the answer, or empty if none
short_answers	The exact answer words from that paragraph, or empty
yes_no_answer	Yes, no, or none

5.5.2 Models

Table 5.3: Models used in SAR, MIGRES, and baseline systems.

Model	Role
sentence-transformers/ multi-qa-mpnet- base-cos-v1 (768-d)	Sentence Transformer [11, 16, 17] used for document, question, information-gap, and evidence embeddings in SAR. The same model is used for SAR cosine-similarity scoring and the Sentence Transformer Search baseline.
gemini-3.5-flash-lite	Language model [18] used by MIGRES for evidence extraction, information-gap detection, sub-query generation, and final answer generation. The same model and call settings are used across SAR and all baselines.
BAAI/bge-reranker-base	Cross-encoder reranker [19, 20] used by MIGRES to filter and rerank retrieved passages after each retrieval step.

5.6 Experimental Design

The experiments evaluate State Aware Retrieval from several perspectives. The study includes hyperparameter selection, comparison with existing retrieval methods, ablation testing, retrieval performance evaluation, iterative state analysis, downstream RAG and robustness evaluation, and computational cost analysis. The validation data are used to tune the scoring weights, and the selected weights are then fixed before the final test evaluation.

5.6.1 Hyperparameter Selection

SAR scores each document using three signals: question relevance, gap coverage, and evidence novelty. The weights α, β, and γ (Table 5.2) set how much each signal counts.

The best balance between the three signals is not known in advance. The weights are non-negative and sum to one (Eq. 8), so only two values are free. They are selected before all other experiments using Bayesian optimization [12], a search method that learns from earlier trials and chooses the next weights that look most promising. It suits this task because each trial needs a full MIGRES run, so only a few trials are affordable.

- **Trials:** 10 in total, where the first 3 use random weights and the next 7 are proposed by the optimizer.
- **One trial:** SAR is run within MIGRES with the candidate weights on 8 validation questions from each of the three datasets, and the trial is scored by the mean Recall@K.
- **Seed:** one fixed random seed is used, so that the search can be repeated.
- **Result:** the weights with the best score are selected, fixed, and used in all later experiments.

Only validation data are used, so the evaluation questions do not influence the weights.

5.6.2 Baseline Comparison

State Aware Retrieval is compared with four established retrieval methods, listed in Table 5.4. All methods use the same document corpus, evaluation questions, and retrieval depth. Each baseline replaces the SAR retrieval step inside the same MIGRES loop. When MIGRES sends several queries, the highest score of each document is kept before the common Top-K selection and filtering steps. Only SAR uses the Sentence Transformer model; the baselines use their own representations. Other retrieval families, such as learned sparse retrieval [21] and late interaction [22, 23], are reviewed in Chapter 2 but are not used as baselines.

Table 5.4: Information Retrieval Baseline Comparison

System	Type	Main Retrieval Basis
BM25	Sparse lexical	Term frequency, inverse document frequency, and document length [9]
DPR	Dense	Question and passage embeddings learned with a dual encoder [8]
Contriever	Dense	Unsupervised dense representations for semantic retrieval [24]
MMR	Lexical, diversity-aware	BM25 candidates re-ranked by maximal marginal relevance [13]
State Aware Retrieval	multi-signal dense retrieval	Question relevance, gap coverage, and evidence novelty

5.6.3 Ablation Study

The ablation study measures the contribution of each signal used by State Aware Retrieval. The full SAR configuration is compared with versions in which individual signals or the weight tuning are removed.

- **SAR-Full:** question relevance, gap coverage, and evidence novelty, with the weights selected by Bayesian optimization.
- **SAR-Without-Relevance:** removes question relevance.
- **SAR-Without-Gap:** removes gap coverage.
- **SAR-Without-Evidence:** removes evidence novelty.
- **SAR-Without-Tuning:** keeps all three signals but does not use the optimized weights. Equal weights $\alpha = \beta = \gamma = \frac{1}{3}$ are used instead.
- **Question-Only:** uses only the similarity between the document and the original question.

5.6.4 Retrieval Performance Evaluation

The retrieval experiment evaluates the quality of the documents selected by State Aware Retrieval. The SAR configuration fixed after validation is compared with the baselines on the raw ranking of each retriever, before the MIGRES filter is applied. For each question, the Top-K documents are checked against the relevant documents of the dataset using standard ranking metrics [25], and each metric is averaged over all evaluation questions.

- **Recall@K:** the share of the relevant documents that appear in the Top-K. It shows how much of the needed evidence is retrieved.
- **Precision@K:** the share of the Top-K documents that are relevant. It shows how clean the retrieved set is.

- **Hit@K:** whether at least one relevant document appears in the Top-K (1 if yes, 0 if no).
- **Mean Reciprocal Rank (MRR):** the average of one divided by the rank of the first relevant document, taken as 0 if none is retrieved. A higher value means that a relevant document appears earlier.
- **nDCG@K:** how good the ranking order is. Relevant documents placed higher count more, and the score is compared with the best possible order [26].

5.6.5 Iterative State Analysis

State Aware Retrieval is intended to change its retrieval behavior as the evidence and information gaps change. This experiment studies the retrieved documents across successive iterations. In the results, iterations are counted from 1, so the first iteration corresponds to $t = 0$ in Section 5.3, and the comparison between consecutive iterations starts from the second iteration.

- **Cumulative recall:** the share of the relevant documents found up to the current iteration.
- **Marginal recall gain:** the additional relevant documents found at the current iteration compared with the previous iterations.
- **New document rate:** the share of the documents retrieved at the current iteration that were not retrieved in any earlier iteration.
- **Retrieval overlap:** how much the documents retrieved at the current iteration overlap with those of the previous iteration.

Together, these measures show whether the changing evidence and gaps lead to different and complementary retrieval results at each iteration.

5.6.6 Downstream RAG and Robustness Evaluation

State Aware Retrieval is evaluated within the MIGRES framework to measure its effect on the final answer. SAR and the baselines use the same MIGRES configuration and language model. The common MIGRES filter is applied after every retriever, so the answer metrics reflect the filtered evidence. The number of documents kept by the filter is recorded separately, because the filter can change which of the retrieved documents reach the answer step.

Answer Evaluation

Answer quality is measured with the official metrics of each dataset where available. Questions without gold answers are excluded.

- **Exact Match (EM):** the share of answers that exactly match a gold answer.
- **Accuracy:** the share of correct answers, checked by normalized exact match against the gold answers and their aliases. No substring match or language-model judge is used.
- **Token-level F1:** the word overlap between the predicted answer and the gold answer, which gives partial credit to partly correct answers [27].

RGB Robustness

The Retrieval-Augmented Generation Benchmark (RGB) tests robustness in four settings [10]:

- **Noise robustness:** answer accuracy when irrelevant documents are mixed into the retrieved set, at different noise levels.
- **Negative rejection:** how often the system correctly refuses to answer when the documents contain no answer.
- **Information integration:** answer accuracy on questions that need information from several documents.
- **Counterfactual robustness:** how often the system detects wrong information in the documents (error detection) and corrects it (error correction).

5.6.7 Computational Cost Analysis

SAR scores every document in the corpus at each iteration, so it may need more computation than a method that scores only a candidate list. SAR computes exact scores for all documents instead of using an approximate nearest-neighbor index [28]. This experiment measures that extra cost. SAR and the baselines run in the same MIGRES loop with the same corpus, the same K, and the same language model settings, so any difference in cost comes from the retrieval method alone.

- **Index build time:** the one-time time needed to embed the corpus and store the document vectors.
- **Retrieval latency:** the average time taken by the retrieval step per iteration, split into the time to embed the question, gaps, and evidence and the time to score the documents.
- **Scoring workload:** the number of documents scored and the number of similarity calculations per question.
- **Language model usage:** the number of language model calls and tokens per question.

5.7 Chapter Solution Summary

This chapter presented State Aware Retrieval (SAR) as a direct retrieval method within the MIGRES framework. SAR uses the current question, information gaps, and collected evidence to guide document selection from the complete document corpus.

The documents are represented in a common Sentence Transformer embedding space. At each iteration, the documents are compared with the current retrieval state, and the final score is used to select the Top-K documents. The selected documents are then passed to MIGRES for filtering, evidence extraction, and state update.

SAR scores the complete corpus directly, with no fixed retrieval threshold. The scoring formulation is defined in Section 5.3, and its weights are selected using validation data. The experiments evaluate the contribution of each state signal, compare SAR with existing retrieval methods, measure retrieval and downstream RAG performance, study its behavior across iterations, and measure its computational cost. The next chapter lists the software and hardware needed to carry out this work.

CHAPTER 6

SOFTWARE AND HARDWARE REQUIREMENTS

The proposed solution uses a Python-based environment for dataset preparation, document embedding, State Aware Retrieval, hyperparameter selection, MIGRES integration, and evaluation. Jupyter Notebook [29] is used for experiment development and analysis, and LaTeX is used to prepare this report. The main software and hardware requirements are described below.

6.1 Software Requirements

Table 6.1: Software Requirements

Software / Tool	**Version**	**Purpose**
Python	3.12.10	Main programming environment for data processing, document embedding, retrieval, and evaluation.
Visual Studio Code	1.131	Experiment development, execution, and result analysis.
PyTorch [30]	2.13.0	Tensor computation and neural model execution.
Sentence Transformers [11]	5.6.1	Generates semantic embeddings for questions, documents, information gaps, and evidence using `multi-qa-mpnet-base-cos-v1`.
Bayesian Optimization [31]	3.3.0	Selects the State Aware Retrieval scoring hyperparameters using validation data.
LaTeX	Overleaf	Preparation of the research document.
Gemini API [18]	gemini-3.5-flash-lite	Provides the language model used by MIGRES for information-gap processing and response generation.

6.3 Hardware Requirements

The proposed system can be implemented on a standard computer with internet access. The main hardware requirements are:

- **CPU:** Intel Core i5 (or equivalent) or higher multi-core processor.
- **RAM:** At least 16 GB.
- **Storage:** At least 20 GB of free SSD space.
- **GPU:** CUDA-compatible NVIDIA GPU with at least 8 GB of VRAM.
- **Network:** Stable internet connection for the language model API and pretrained model downloads.

CHAPTER 7

IMPLEMENTATION

7.1 Introduction

This chapter describes the implementation of the proposed SAR-based retrieval system within the MIGRES framework. It first presents the development environment, including the hardware and software used. It then explains how the system is organized and integrated, covering the document corpus and embedding pipeline, the MIGRES module, and the SAR retrieval module. The SAR algorithm is then given in pseudocode, and the chapter ends with the workflow used to execute the prepared experiments.

7.2 Development Environment

7.2.1 Hardware Environment

The experiments are executed using a Google Colab managed runtime with an NVIDIA Tesla T4 GPU. The T4 provides 16 GB of GDDR6 GPU memory and 2,560 CUDA cores. Google Colab provides the CPU, system memory, and temporary runtime storage as part of the managed notebook environment, with the exact allocation depending on the runtime instance. The experiments therefore use the following compute environment.

Table 7.1: Hardware Environment

Component	**Specification**
Execution Environment	Google Colab managed runtime
GPU	NVIDIA Tesla T4
GPU Memory	16 GB GDDR6
CUDA Cores	2,560
CPU	Colab-managed virtual CPU
System Memory	Colab-managed runtime memory
Storage	Temporary runtime storage provided by Colab
Internet	Required for model downloads, dataset access, and language model API calls

7.2.2 Software Environment

The implementation is developed in Python using Visual Studio Code, while the experiments are executed through Jupyter notebooks in Google Colab. Similarity computations use NumPy [32], and

the Sentence Transformers library builds on the Hugging Face Transformers library [33]. The main software components used in the experimental pipeline are summarized in Table 7.2.

Table 7.2: Software Environment

Software / Tool	**Purpose**
Python	Core implementation and experiment pipeline
Visual Studio Code	Development and code management
Jupyter Notebook	Execution and analysis of experiments
PyTorch	Model inference and tensor computation
Sentence Transformers	Document and retrieval-state embeddings
Bayesian Optimization	Selection of SAR weighting parameters
MIGRES Framework	Iterative retrieval and evidence processing
Google Gemini API	Language model operations within MIGRES
Local Disk-backed Vector Store	Storage of precomputed document embeddings
YAML Configuration	Management of model, retrieval, and experiment settings

7.3 System Implementation

The proposed system is implemented by integrating the SAR retrieval method into the existing MIGRES framework. The implementation is organized into three main components that work together during retrieval. The document corpus and embedding module first prepares the searchable corpus by processing the documents, generating their embeddings, and storing the resulting vectors in the vector store. This preparation is completed before the experiments are executed. During retrieval, MIGRES manages the question-answering process and maintains the current retrieval state, while the SAR module uses the query and state information together with the precomputed document vectors to calculate relevance, gap coverage, and evidence novelty and select the Top-K documents. The retrieved documents are then returned to MIGRES for filtering, evidence extraction, and state updating before the next iteration.

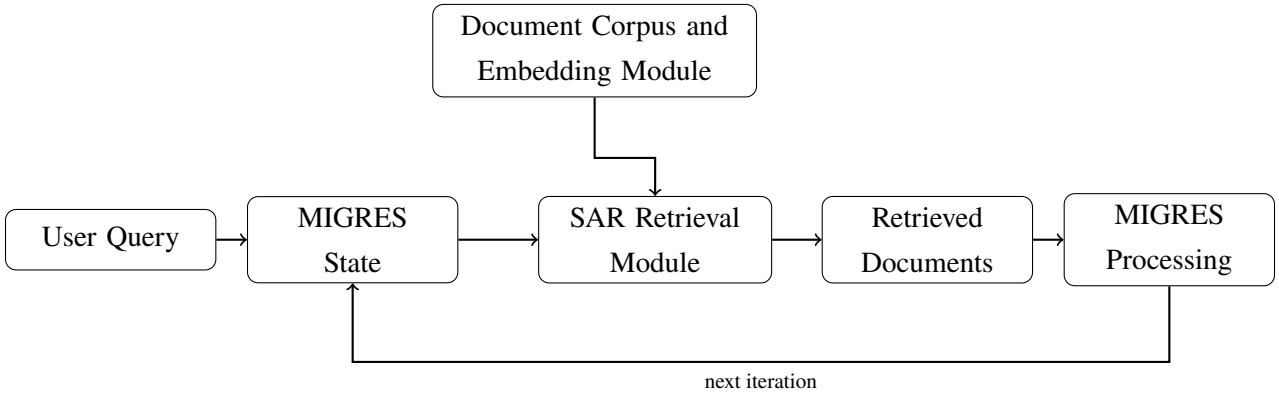


Figure 7.1: Overall System Implementation Flow

7.3.1 Document Corpus and Embedding Module

This module prepares the document collection used by the SAR retriever. Its purpose is to convert the source datasets into a common searchable format and create the vector representations required for retrieval. The module performs four main stages: dataset preparation, passage chunking, document embedding, and vector storage. The resulting vector store is then used by SAR during document scoring and Top-K selection.

The module consists of the following stages:

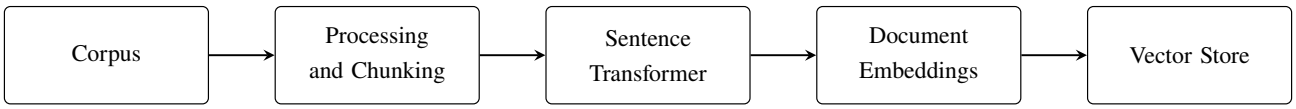


Figure 7.2: Corpus Preparation and Embedding Pipeline

7.3.2 MIGRES Module

MIGRES is the base framework that manages the iterative question-answering process. It maintains the information required at each iteration and controls how retrieved documents are processed until the question can be answered or the stopping condition is reached.

In this project, MIGRES provides the main execution flow, while SAR is integrated as the retrieval method within that flow. The MIGRES implementation can be viewed through two main parts: the retrieval state maintained during each iteration and the processing stage that handles the retrieved documents and updates this state.

The MIGRES state contains the original question q, the evidence collected up to iteration t, denoted by E_t, and the remaining information gaps G_t. This state is provided to the retrieval stage for each iteration. After retrieval, MIGRES processes the returned documents through filtering and evidence extraction. It then checks whether sufficient information is available. If further information is required, new queries are generated, the state is updated, and the next retrieval iteration begins.

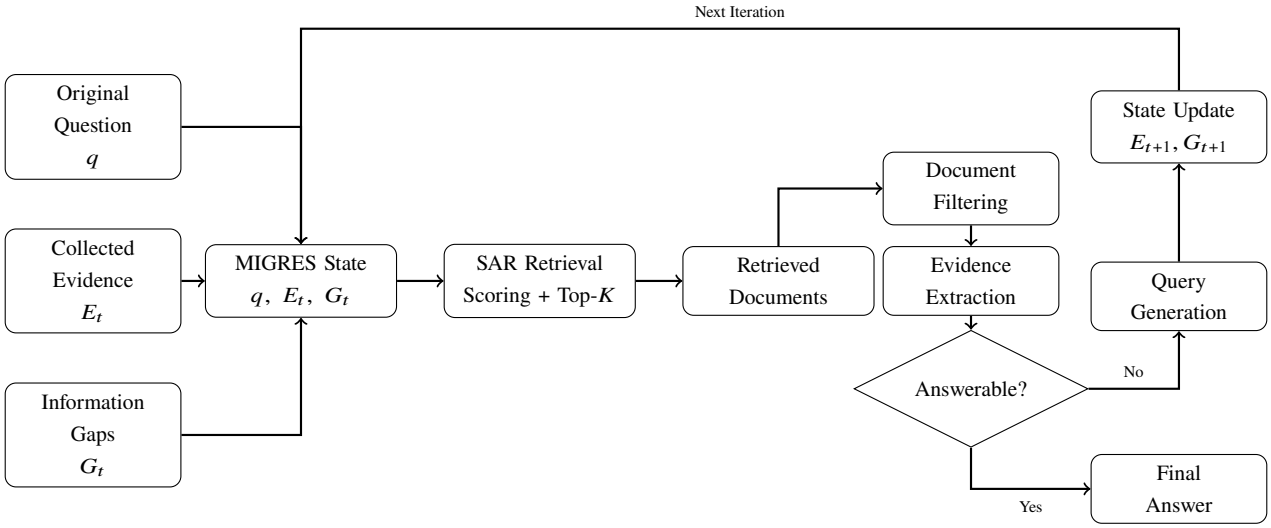


Figure 7.3: MIGRES State and Processing Flow with SAR Integration

7.3.3 SAR Module

State-Aware Retrieval (SAR) is the retrieval method proposed in this project. It selects documents based on the current retrieval state rather than using only the original question. The state consists of the question q, the collected evidence E_t, and the remaining information gaps G_t.

SAR uses three components to score each candidate document: question relevance, gap coverage, and evidence novelty. These components measure how relevant a document is to the question, how well it addresses the remaining information gaps, and how much new information it provides beyond the evidence already collected. The three scores are combined using the SAR weighting formulation defined in Chapter 5.

The resulting SAR score is used to rank the candidate documents, and the highest-ranked K documents are selected as the retrieval output. In this implementation, $K = 50$. The selected documents are then returned to MIGRES for further processing. The overall SAR retrieval flow is shown in Figure 7.4.

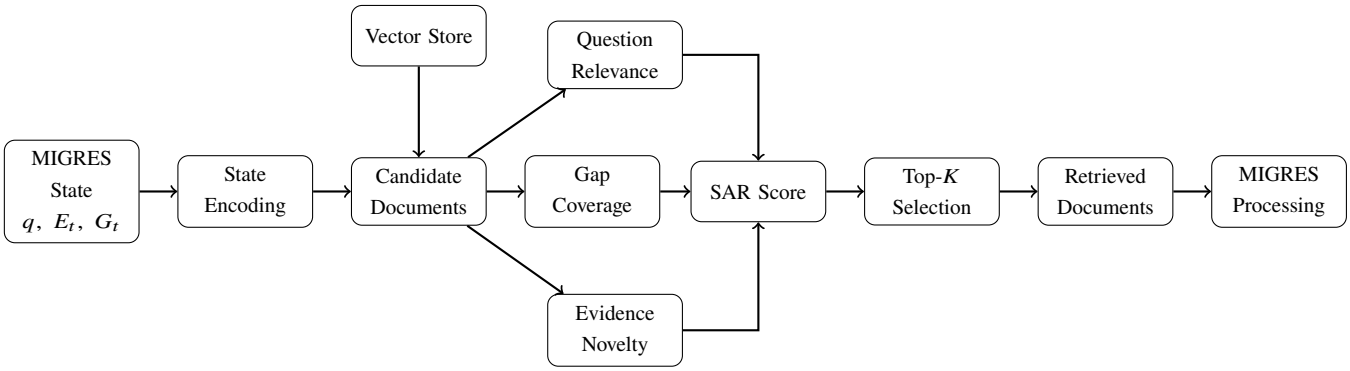


Figure 7.4: Architecture of the SAR Retrieval Module

7.4 Experimental Setup and Execution Plan

The experimental setup defines how the proposed system is configured, executed, and evaluated in practice. The experiments involve connecting the implemented modules into complete execution pipelines, preparing the required inputs and configurations, running the defined experimental conditions, and recording the outputs produced by each run. The setup is designed to ensure that the same procedures can be executed consistently across different experimental configurations, while preserving the results and intermediate information required for subsequent analysis and comparison.

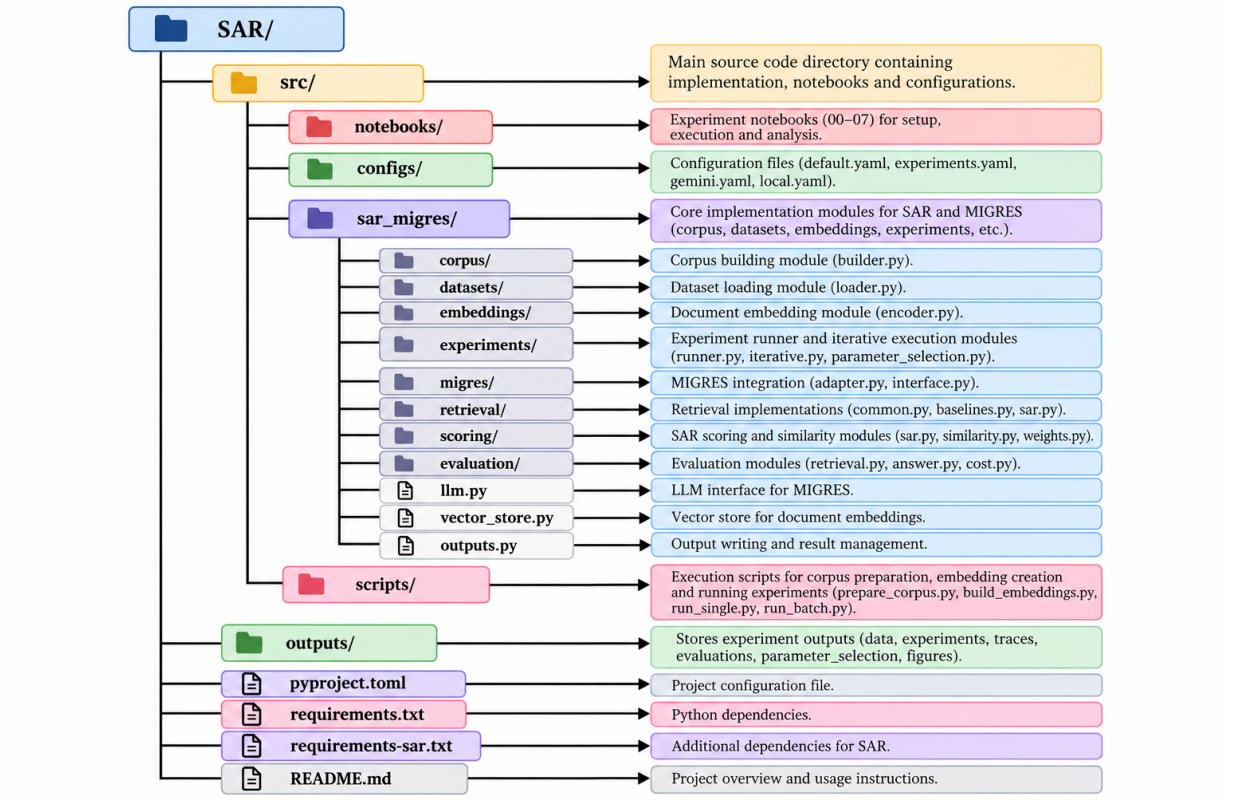


Figure 7.5: Project Structure of the SAR–MIGRES Implementation

7.4.1 Integration of MIGRES with State-Aware Retrieval

The official MIGRES code is used as it is, without any change to its files. The connection between MIGRES and SAR is made in one file, `adapter.py`. This file does the following:

- **Loads MIGRES.** MIGRES has a class called `RetGBatched` in its file `pipeline.py`, which controls the retrieval steps. `adapter.py` reads this class so that it can be used in our project.
- **Creates a new class.** `adapter.py` creates a new class based on `RetGBatched`. The new class works exactly like the original one, except for two functions that are replaced.
- **Replaces `search()`.** MIGRES calls this function whenever it needs documents. In the new class, it sends the current question, the evidence collected so far, and the missing information to the SAR retriever instead of the MIGRES searcher.

- **Replaces `askGPT()`.** MIGRES calls this function to get a response from the language model. In the new class, it sends the prompt to `llm.py`, which calls the model and returns the response.

The retrieval is done in two files. In `sar.py` (retrieval), the function `retrieve()` of **StateAwareRetriever** receives the question, evidence, and missing information from `search()`. It scores the documents using `score_components()` and `sar_score()` from `sar.py` (scoring) and selects the Top-K documents. These are returned to MIGRES through `adapter.py`, and MIGRES continues its iterative process as usual. Therefore, SAR replaces only the document retrieval step of MIGRES, and the rest of the iterative process is unchanged.

7.4.2 Code-Level Architecture

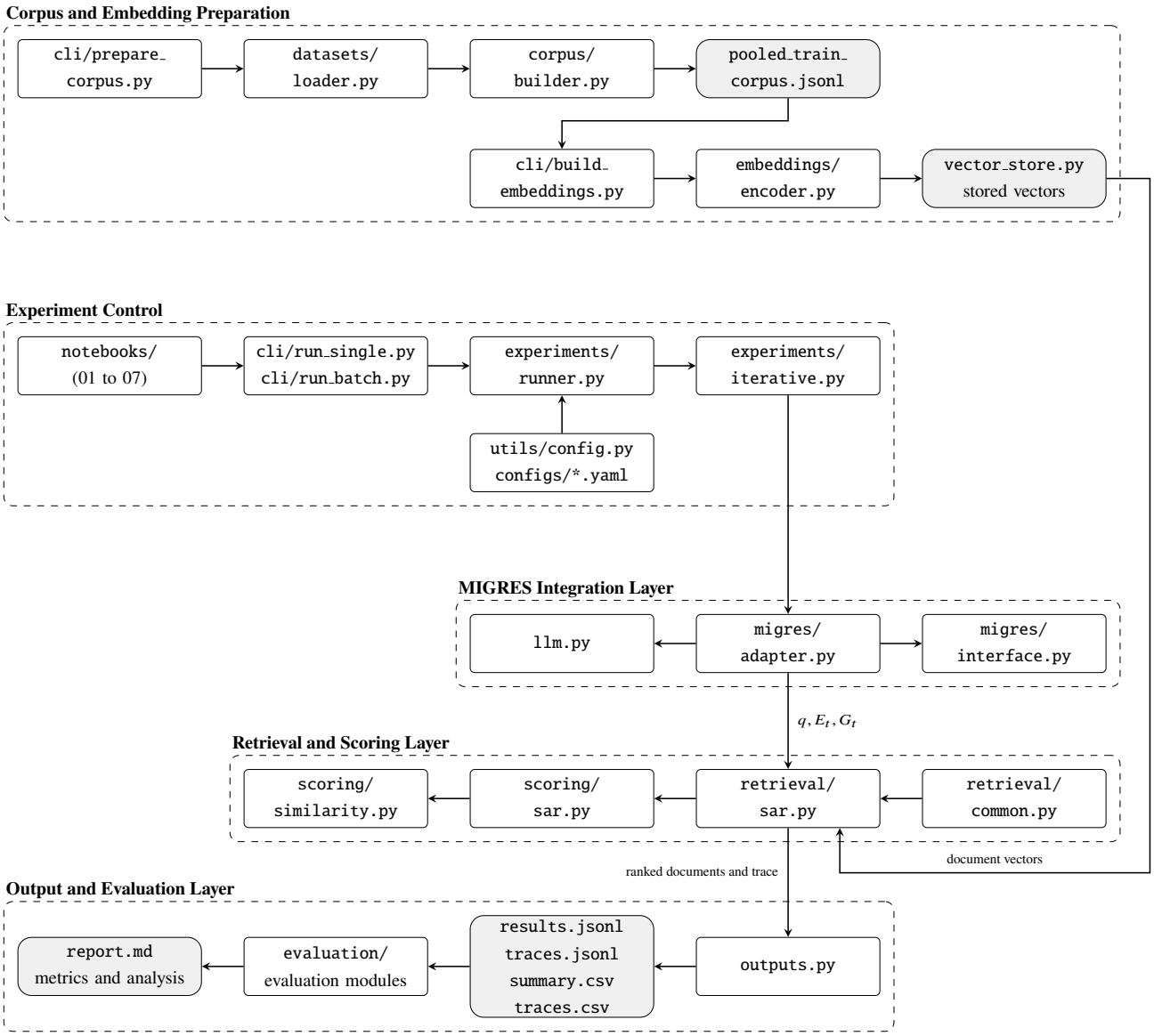


Figure 7.6: Code-Level Architecture of the SAR–MIGRES Implementation

In summary, the code-level architecture maps the SAR–MIGRES design onto the `sar_migres` package, where each module has a single responsibility. SAR is connected to MIGRES only through the adapter, so the rest of the iterative pipeline remains unchanged.

7.4.3 SAR Scoring and Retrieval Module

The SAR scoring and retrieval module decides which documents are returned to MIGRES at each iteration. The scoring module (`scoring/sar.py`, `scoring/similarity.py`) evaluates each document against the current retrieval state, and the retrieval module (`retrieval/sar.py`) applies this scoring to the whole corpus and selects the Top-K documents.

Given the question q, the information gaps G_t, the collected evidence E_t, and the stored document vectors, the scoring module computes three signals using the normalized cosine similarity $s(x, y) = \frac{1+\cos(x,y)}{2}$:

$$R(d,q) = s(d,q), \qquad C(d,G_t) = \frac{1}{|G_t|}\sum_{g_i \in G_t} s(d,g_i), \qquad N(d,E_t) = 1 - \max_{e_j \in E_t} s(d,e_j),$$

where $C = 0$ if G_t is empty and $N = 1$ if E_t is empty. The final score is

$$S_t(d) = \alpha R(d,q) + \beta C(d,G_t) + \gamma N(d,E_t),$$

with default weights $\alpha = 0.4$, $\beta = 0.3$, $\gamma = 0.3$.

The retrieval module scores the document vectors in chunks and returns the $K = 50$ highest-scoring documents, with ties broken by document id. The ranked documents and their scores are passed to MIGRES through `adapter.py` and saved in the retrieval trace.

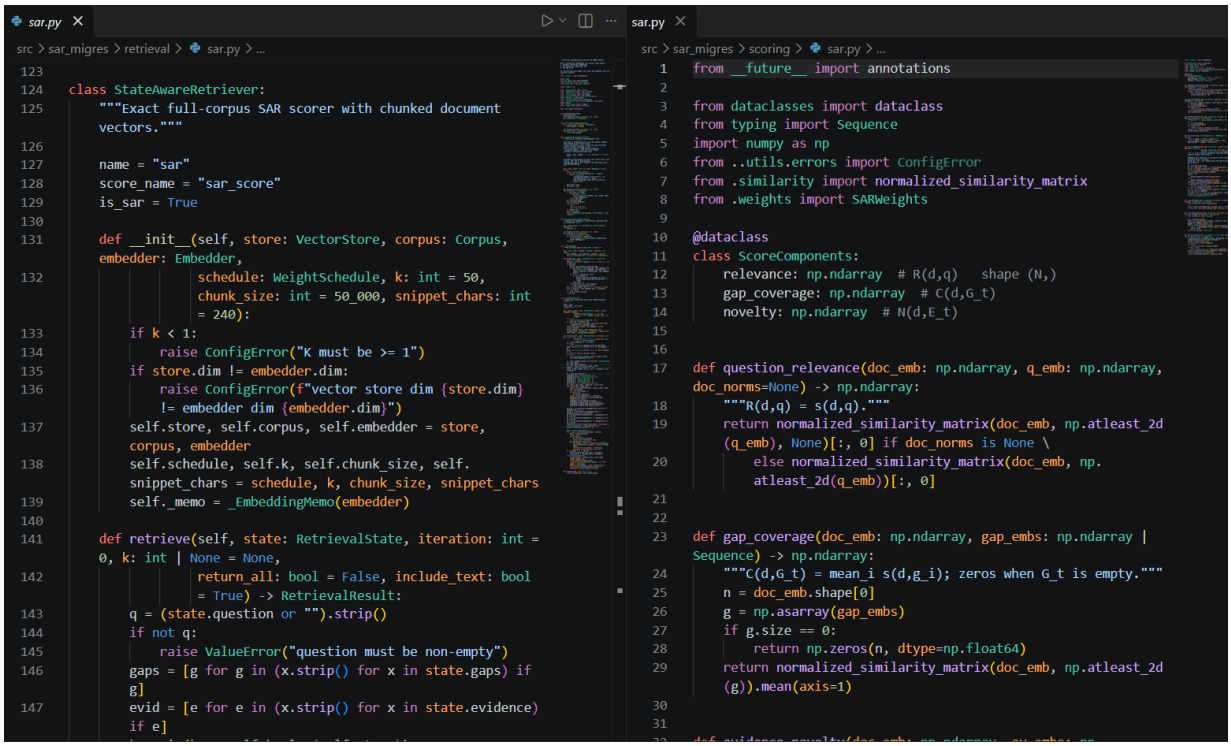


Figure 7.7: Implementation of the SAR Scoring and Retrieval Module

The following pseudocode shows how **score_components()**, **sar_score()**, and **StateAwareRetriever.retrieve()** work together.

Algorithm 7.1: SAR Scoring and Retrieval

Input: question q, gaps G_t, evidence E_t, document vector store $\mathcal{V}$, weights (α, β, γ), depth K
Output: ranked Top-K documents T_t with R, C, N, S_t

Procedure ScoreChunk(B, q, G_t, E_t)
$S \leftarrow s(B, [q; G_t; E_t])$ using one matrix product
$R \leftarrow$ column of S for q
if $G_t = \emptyset$ **then** $C \leftarrow 0$ **else** $C \leftarrow$ mean of the gap columns of S
if $E_t = \emptyset$ **then** $N \leftarrow 1$ **else** $N \leftarrow 1-$ max of the evidence columns of S
$S_t \leftarrow \alpha R + \beta C + \gamma N$
return (R, C, N, S_t)

Procedure Retrieve$(q, G_t, E_t, \mathcal{V}, K)$
Remove empty strings from G_t and E_t
$K \leftarrow \min(K, |\mathcal{V}|)$
Encode q, G_t, and E_t
$L \leftarrow \emptyset$
for each chunk B of document vectors in $\mathcal{V}$ **do**
    $(R, C, N, S_t) \leftarrow$ ScoreChunk(B, q, G_t, E_t)
    Add the K highest-scoring documents of B to L (ties broken by document id)
end for
Sort L by S_t (descending), then by document id
$T_t \leftarrow$ first K documents of L
return T_t

In summary, the scoring module turns the retrieval state into a score for each document, and the retrieval module uses these scores to select the documents that MIGRES receives at the next iteration.

7.4.4 Notebooks Used for Experimentation

The experiments are executed in Google Colab through notebooks that serve as thin interfaces to the shared implementation in `src/sar_migres`. The notebooks do not contain the retrieval algorithm. They call the same modules for SAR and for the baseline methods, which ensures that all methods are run and recorded under identical conditions. Notebook 00 prepares the Colab environment, and notebooks 01 to 07 correspond to the experiments described in Section 5.6. The experimental settings, including the SAR weights, datasets, MIGRES parameters, and LLM configuration, are specified in the YAML files under `src/configs/`.

Table 7.3: Experiment Notebooks and Their Mapping to the Experimental Design

No.	Notebook	Experiment (Section 5.6)	Purpose
00	00_colab_setup.ipynb	Not an experiment	Sets up the environment for Google Colab execution.
01	01_parameter_hyperparameter_selection.ipynb	5.6.1 Hyperparameter Selection	Selects the SAR weighting parameters.
02	02_baseline_comparison.ipynb	5.6.2 Baseline Comparison	Compares SAR with the baseline retrievers.
03	03_ablation_study.ipynb	5.6.3 Ablation Study	Evaluates the contribution of each SAR component.
04	04_retrieval_performance.ipynb	5.6.4 Retrieval Performance Evaluation	Calculates retrieval metrics from completed runs.
05	05_iterative_state_analysis.ipynb	5.6.5 Iterative State Analysis	Analyzes retrieval behavior across iterations using the stored traces.
06	06_downstream_rag_robustness.ipynb	5.6.6 Downstream RAG and Robustness Evaluation	Evaluates downstream answer performance and robustness.
07	07_computational_cost.ipynb	5.6.7 Computational Cost Analysis	Measures retrieval and execution cost.

7.4.5 Execution Output and Result Format

Every experimental run saves its outputs to a separate run folder inside the output directory of its experiment, for example outputs/experiments/02_baseline_comparison/runs/<run_id>/. The files are written during execution and are stored as JSON, JSONL, and CSV files for further processing, as a Markdown report for reading, and as a log file for the run. Question-level results and iteration-level retrieval traces are stored separately, so answer analysis and retrieval analysis can be performed independently.

Parameter selection saves two additional files in outputs/experiments/01_parameter_selection/. The first is the selected SAR weights, which are frozen and read by the later experiments. The second is the log of all tuning trials.

The evaluation metrics are not saved as separate files. They are calculated by the evaluation modules from the saved run files and shown in the notebook output. These include the retrieval metrics, iterative analysis, answer metrics, and computational cost.

Table 7.4: Outputs Saved for Each Experimental Run

Output	**Format**	**Data stored**
Question results	JSONL	Final answer, gold answer, run status, execution time, and LLM usage for each question.
Retrieval traces	JSONL	For each iteration: information gaps, collected evidence, ranked documents with their R, C, N and final scores, SAR weights, and retrieval timing.
Trace table	CSV	Flattened version of the traces with one row per retrieved document, used for tabular analysis.
Summary table	CSV	One row per question with status, number of retrieval iterations, final answer, execution time, and LLM usage.
Configuration and metadata	JSON	Configuration used for the run, together with run information such as the retriever, dataset, corpus, models, and warnings.
Run report	Markdown	Readable summary of the run, with the retrieved documents and scores for each iteration.
Execution log	LOG	Messages written during the run, such as progress, warnings, and errors.

7.5 Conclusion

This chapter described the implementation of the proposed SAR-MIGRES system and its supporting experimental infrastructure. The SAR retrieval procedure, document embedding and vector-store preparation, MIGRES integration, baseline retrieval methods, and evaluation components have been implemented in the shared Python source and experiment notebooks.

The system is now prepared for the final experimental runs. The implementation and evaluation pipeline are in place, while the final execution and generation of experimental results remain pending.

CHAPTER 8

APPLICATIONS

1. **Complex Question Answering:** SAR can support AI systems in answering complex questions where the required information is spread across multiple documents and cannot be obtained in a single retrieval step.

2. **Multi-Step Information Retrieval:** SAR can be applied to systems that search for information repeatedly, helping each retrieval step focus on the information that has not yet been obtained.

3. **Research and Knowledge Discovery:** SAR can help collect information from large document collections by prioritizing useful and complementary evidence instead of repeatedly retrieving similar content.

4. **Evidence-Based RAG Applications:** SAR can be integrated into RAG-based applications that need to build a complete set of evidence before generating an answer, such as question answering, document analysis, and knowledge-based assistants.

CHAPTER 9

GANTT CHART

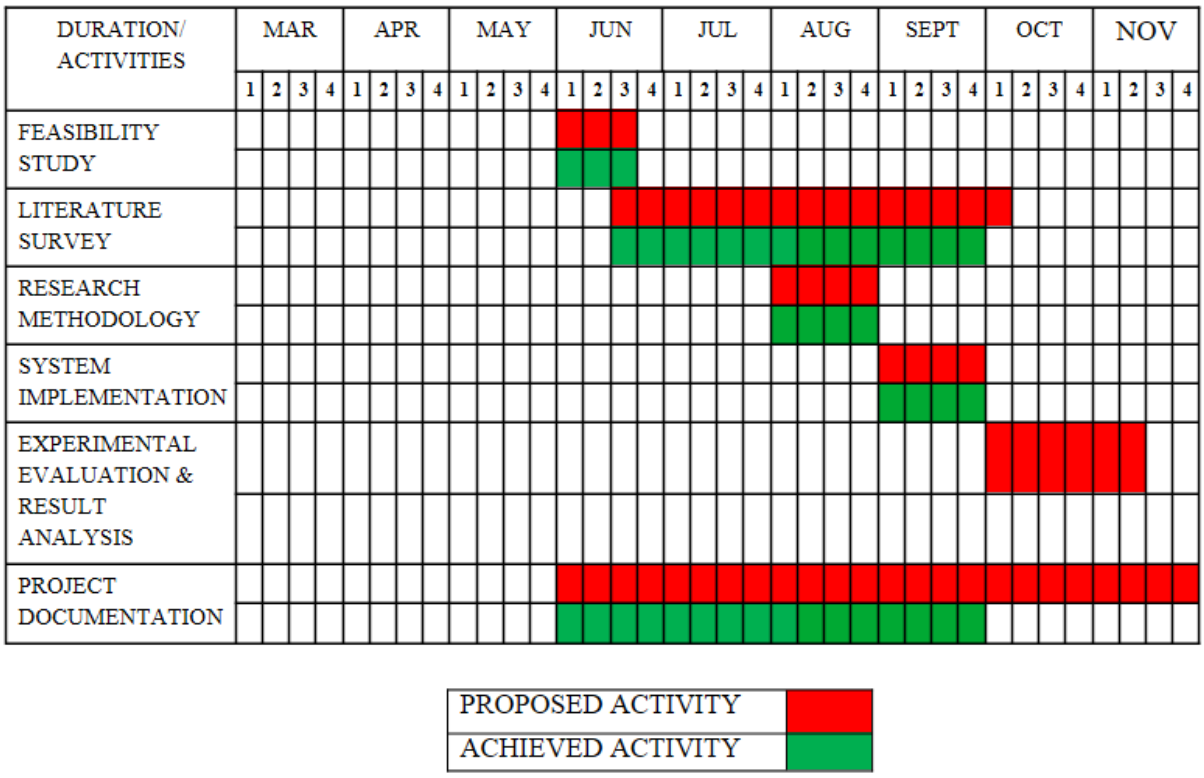


Figure 9.1: Gantt Chart

CHAPTER 10

CONCLUSION

This project presents **State Aware Retrieval (SAR)** as a direct retrieval method within the MIGRES framework. SAR uses the current question, the information gaps, and the previously collected evidence to guide document selection from the complete document corpus. The selected documents are then passed to MIGRES for filtering, evidence extraction, and further state updates.

The study will examine whether making retrieval aware of the current information state can improve the quality and usefulness of the retrieved evidence during iterative retrieval. The contribution of each state signal will be studied through ablation experiments, while comparisons with established retrieval methods will measure the overall retrieval performance of SAR. The effect on final RAG performance, behavior across iterations, robustness, and computational cost will also be evaluated.

Overall, this work provides a clear framework for testing whether State Aware Retrieval can make iterative retrieval more focused on the information that is relevant, still missing, and not already covered by previous evidence. The main MIGRES framework remains unchanged, so the study can directly examine the effect of the proposed retrieval method. The final experimental results will determine the effectiveness and practical value of SAR for iterative retrieval-augmented generation.

REFERENCES

[1] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. K"uttler, M. Lewis, W.-t. Yih, T. Rockt"aschel, S. Riedel, and D. Kiela, "Retrieval-augmented generation for knowledge-intensive nlp tasks," in *Advances in Neural Information Processing Systems 33*, 2020, pp. 9459–9474. [Online]. Available: https://papers.nips.cc/paper/2020/hash/6b493230205f780e1bc26945df7481e5-Abstract.html

[2] Z. Yang, P. Qi, S. Zhang, Y. Bengio, W. Cohen, R. Salakhutdinov, and C. D. Manning, "Hotpotqa: A dataset for diverse, explainable multi-hop question answering," in *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, 2018, pp. 2369–2380.

[3] H. Trivedi, N. Balasubramanian, T. Khot, and A. Sabharwal, "Musique: Multihop questions via single-hop question composition," *Transactions of the Association for Computational Linguistics*, vol. 10, pp. 539–554, 2022.

[4] V. Yadav, S. Bethard, and M. Surdeanu, "Unsupervised alignment-based iterative evidence retrieval for multi-hop question answering," in *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, 2020, pp. 4514–4525.

[5] K. Wang, F. Duan, P. Li, S. Wang, and X. Cai, "Llms know what they need: Leveraging a missing information guided framework to empower retrieval-augmented generation," in *Proceedings of the 31st International Conference on Computational Linguistics*, 2025, pp. 2379–2400.

[6] M. Pickett, J. Hartman, A. K. Bhowmick, R. ul Alam, and A. Vempaty, "Better rag using relevant information gain," *arXiv preprint arXiv:2407.12101*, 2024.

[7] G. Izacard and E. Grave, "Leveraging passage retrieval with generative models for open domain question answering," in *Proceedings of the 16th Conference of the European Chapter of the Association for Computational Linguistics: Main Volume*, 2021, pp. 874–880.

[8] V. Karpukhin, B. Oguz, S. Min, P. Lewis, L. Wu, S. Edunov, D. Chen, and W. tau Yih, "Dense passage retrieval for open-domain question answering," in *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing*, 2020, pp. 6769–6781.

[9] S. Robertson and H. Zaragoza, "The probabilistic relevance framework: Bm25 and beyond," *Foundations and Trends in Information Retrieval*, vol. 3, no. 4, pp. 333–389, 2009.

[10] J. Chen, H. Lin, X. Han, and L. Sun, "Benchmarking large language models in retrieval-augmented generation," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 38, no. 16, 2024, pp. 17 754–17 762.

[11] N. Reimers and I. Gurevych, "Sentence-bert: Sentence embeddings using siamese bert-networks," in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing*, 2019, pp. 3982–3992.

[12] J. Snoek, H. Larochelle, and R. P. Adams, "Practical bayesian optimization of machine learning algorithms," in *Advances in Neural Information Processing Systems*, vol. 25, 2012.

[13] J. Carbonell and J. Goldstein, "The use of MMR, diversity-based reranking for reordering documents and producing summaries," in *Proceedings of the 21st Annual International ACM SIGIR Conference on Research and Development in Information Retrieval*, 1998, pp. 335–336.

[14] R. Agrawal, S. Gollapudi, A. Halverson, and S. Ieong, "Diversifying search results," in *Proceedings of the Second ACM International Conference on Web Search and Data Mining*, 2009, pp. 5–14.

[15] T. Kwiatkowski, J. Palomaki, O. Redfield, M. Collins, A. Parikh, C. Alberti, D. Epstein, I. Polosukhin, J. Devlin, K. Lee, K. Toutanova, L. Jones, M. Kelcey, M.-W. Chang, J. Uszkoreit, Q. Le, and S. Petrov, "Natural questions: A benchmark for question answering research," *Transactions of the Association for Computational Linguistics*, vol. 7, pp. 452–466, 2019.

[16] K. Song, X. Tan, T. Qin, J. Lu, and T.-Y. Liu, "MPNet: Masked and permuted pre-training for language understanding," in *Advances in Neural Information Processing Systems*, vol. 33, 2020, pp. 16 857–16 867.

[17] Sentence Transformers, "multi-qa-mpnet-base-cos-v1," Hugging Face model card, 2020, 768-dimensional sentence-transformers model designed for semantic search. [Online]. Available: https://huggingface.co/sentence-transformers/multi-qa-mpnet-base-cos-v1

[18] Gemini Team, Google, "Gemini: A family of highly capable multimodal models," *arXiv preprint arXiv:2312.11805*, 2023.

[19] R. Nogueira and K. Cho, "Passage re-ranking with BERT," *arXiv preprint arXiv:1901.04085*, 2019.

[20] S. Xiao, Z. Liu, P. Zhang, N. Muennighoff, D. Lian, and J.-Y. Nie, "C-Pack: Packed resources for general Chinese embeddings," in *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval*, 2024, pp. 641–649.

[21] T. Formal, C. Lassance, B. Piwowarski, and S. Clinchant, "SPLADE: Sparse lexical and expansion model for first stage ranking," *arXiv preprint arXiv:2107.05720*, 2021.

[22] O. Khattab and M. Zaharia, "ColBERT: Efficient and effective passage search via contextualized late interaction over BERT," in *Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval*, 2020, pp. 39–48.

[23] K. Santhanam, O. Khattab, J. Saad-Falcon, C. Potts, and M. Zaharia, "Colbertv2: Effective and efficient retrieval via lightweight late interaction," in *Proceedings of the 2022 Conference of the North American Chapter of the Association for Computational Linguistics*, 2022, pp. 3715–3734.

[24] G. Izacard, M. Caron, L. Hosseini, S. Riedel, P. Bojanowski, A. Joulin, and E. Grave, "Unsupervised dense information retrieval with contrastive learning," in *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*, 2022, pp. 2665–2679.

[25] C. D. Manning, P. Raghavan, and H. Schütze, *Introduction to Information Retrieval*. Cambridge University Press, 2008.

[26] K. Järvelin and J. Kekäläinen, "Cumulated gain-based evaluation of ir techniques," *ACM Transactions on Information Systems*, vol. 20, no. 4, pp. 422–446, 2002.

[27] P. Rajpurkar, J. Zhang, K. Lopyrev, and P. Liang, "SQuAD: 100,000+ questions for machine comprehension of text," in *Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing*, 2016, pp. 2383–2392.

[28] J. Johnson, M. Douze, and H. Jégou, "Billion-scale similarity search with gpus," *arXiv preprint arXiv:1702.08734*, 2017.

[29] T. Kluyver, B. Ragan-Kelley, F. Pérez, B. Granger, M. Bussonnier, J. Frederic, K. Kelley, J. Hamrick, J. Grout, S. Corlay, P. Ivanov, D. Avila, S. Abdalla, and C. Willing, "Jupyter notebooks – a publishing format for reproducible computational workflows," in *Positioning and Power in Academic Publishing: Players, Agents and Agendas*. IOS Press, 2016, pp. 87–90.

[30] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, A. Desmaison, A. Köpf, E. Yang, Z. DeVito, M. Raison, A. Tejani, S. Chilamkurthy, B. Steiner, L. Fang, J. Bai, and S. Chintala, "PyTorch: An imperative style, high-performance deep learning library," in *Advances in Neural Information Processing Systems 32*, 2019, pp. 8024–8035.

[31] F. Nogueira, "Bayesian optimization: Open source constrained global optimization tool for Python," Software package, 2014. [Online]. Available: https://github.com/bayesian-optimization/BayesianOptimization

[32] C. R. Harris, K. J. Millman, S. J. van der Walt, R. Gommers, P. Virtanen, D. Cournapeau, E. Wieser, J. Taylor, S. Berg, N. J. Smith, R. Kern, M. Picus, S. Hoyer, M. H. van Kerkwijk, M. Brett, A. Haldane, J. F. del Río, M. Wiebe, P. Peterson, P. Gérard-Marchant, K. Sheppard, T. Reddy, W. Weckesser, H. Abbasi, C. Gohlke, and T. E. Oliphant, "Array programming with NumPy," *Nature*, vol. 585, no. 7825, pp. 357–362, 2020.

[33] T. Wolf, L. Debut, V. Sanh, J. Chaumond, C. Delangue, A. Moi, P. Cistac, T. Rault, R. Louf, M. Funtowicz, J. Davison, S. Shleifer, P. von Platen, C. Ma, Y. Jernite, J. Plu, C. Xu, T. L. Scao, S. Gugger, M. Drame, Q. Lhoest, and A. M. Rush, "Transformers: State-of-the-art natural language processing," in *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, 2020, pp. 38–45.